

Rapport de Stage Projet de Fin d'année

Conception et développement d'une plateforme AIOps basée sur l'observabilité et l'intelligence artificielle pour la détection d'anomalies, la prédiction et l'assistance à la résolution des incidents informatiques

Réalisé au sein de :



Soutenu le 17/09/2026

Réalisé par :

Hamza Elmourabit

Devant le jury composé de :

Pr.NAFDI AHMED

Pr.Lahcen Moumoun

Pr.Jarrar abdessamad

Encadré par :

Pr.NAFIDI Ahmed

Année Universitaire :

2025/2026

Conception et développement d'une plateforme AIOps fondée sur la télémétrie et l'intelligence artificielle pour la détection d'anomalies, la prédiction et la résolution assistée des incidents informatiques, basée sur le machine learning, NLP et RAG

Remerciements

Je tiens à exprimer ma reconnaissance à toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce Projet de Fin d'Année. Je remercie en premier lieu Monsieur FADIL BADR, encadrant professionnel chez HARDTEC MAROC, pour son accompagnement, ses orientations et les échanges techniques qui ont permis de cadrer progressivement le projet. Je remercie également Pr. Ahmed Nafidi, encadrant académique à l'ENSA Berrechid, pour son suivi pédagogique et ses conseils dans la structuration du travail.

Je remercie les membres du jury, Pr. Abdessamad Jarrar et Pr. Lahcen Moumoun, pour l'attention accordée à ce travail et pour les remarques qui permettront d'en apprécier les apports et les limites. j'adresse mes remerciements à l'équipe de HARDTEC MAROC pour l'accueil professionnel et l'environnement de travail qui ont accompagnés deux mois de stage.

Résumé

Ce rapport présente la conception et le développement de HARDTEC AIOps Intelligent Support Platform, une plateforme orientée opérations IT qui rapproche le traitement intelligent des tickets, l'analyse prédictive d'incidents et l'assistance fondée sur l'historique. Le dépôt étudié met en évidence plusieurs briques complémentaires : un pipeline de classification textuelle des tickets, une couche de données articulée au-tour de Snowake et dbt, une analytique locale avec DuckDB, une persistance SQLite des prédictions, des services FastAPI, une interface Streamlit, des workflows Airflow, des artefacts MLflow et une couche de recherche sémantique basée sur Sentence Trans-formers et FAISS. Un service d'orchestration de support est également présent sous la forme d'une API dédiée, avec validation humaine explicitement conservée.

La partie AIOps s'appuie sur une expérimentation MicroSS/XGBoost visant à estimer le risque d'un incident dans les quinze minutes suivantes à partir de fenêtres temporelles de cinq minutes. La version V3.1 documentée dans le dépôt utilise 115 métriques de base et une construction de 1 380 variables, notamment des retards temporels, variations, pourcentages de variation et statistiques glissantes. Cette approche doit toutefois être interprétée comme une prédiction fondée sur des données historiques et non comme un système de monitoring industriel temps réel. Le dépôt contient le modèle et ses métadonnées, mais pas le jeu de données MicroSS source nécessaire à une exécution complète dans l'environnement extrait.

L'analyse du dépôt conduit également à une distinction essentielle entre ce qui est réellement implémenté, ce qui est présent sous forme expérimentale et ce qui reste à compléter. Les tests disponibles, exécutés dans l'environnement d'analyse, montrent notamment des problèmes d'environnement et trois échecs fonctionnels dans la suite principale : statut API dégradé, dépendance DuckDB absente et incohérence entre le contrat du test du prédicteur et la valeur effectivement retournée par celui-ci. Ces constats sont intégrés aux limites du projet plutôt que masqués.

Mots-clés : AIOps, Data Engineering, Machine Learning, classification de tickets, XGBoost, MicroSS, RAG, embeddings, FAISS, FastAPI, Streamlit, Airflow, Snowflake, dbt, DuckDB, Docker, MLflow.

Abstract

This report presents the design and development of HARDTEC AIOps Intelligent Support Platform, an IT operations-oriented platform combining intelligent ticket processing, incident-risk prediction and historical support assistance. The analyzed repository contains a text-based ticket classification pipeline, a data layer involving Snowflake and dbt, local analytics through DuckDB, SQLite persistence for prediction history, FastAPI services, a Streamlit interface, Airflow workflows, MLflow artifacts, and semantic retrieval based on Sentence Transformers and FAISS. A dedicated support orchestration service is also included, while human approval remains explicitly required for irreversible actions.

The AIOps component uses a MicroSS/XGBoost approach to estimate the probability of an incident during the following fifteen minutes from ve-minute temporal windows. Version V3.1 relies on 115 base metrics and produces 1,380 engineered features including lags, deltas, percentage changes and rolling statistics. This component must be understood as a historical-data prediction experiment rather than as a production-grade real-time monitoring system. The repository includes the model and feature metadata, but the corresponding MicroSS source dataset is not included in the extracted project.

The repository audit also separates implemented capabilities from incomplete or experimental elements. Tests executed during the analysis revealed environment issues and three failures in the main test suite : a degraded API health status, a missing DuckDB dependency, and a mismatch between the predictor test contract and the actual predictor return payload. These findings are reported as engineering limitations rather than being concealed.

Keywords : AIOps, Data Engineering, Machine Learning, ticket classification, XG-Boost, MicroSS, RAG, embeddings, FAISS, FastAPI, Streamlit, Airflow, Snowflake, dbt, DuckDB, Docker, MLflow.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des abréviations	viii
1 Introduction générale	1
1.1 Contexte général	1
1.2 Problématique	1
1.3 Objectifs du projet	2
1.4 Méthodologie et démarche	3
1.5 Organisation du rapport	3
1.6 Périmètre et règle de vérité du dépôt	4
2 Contexte du projet et analyse des besoins	5
2.1 Cadre académique et professionnel	5
2.2 Problème métier et enjeux opérationnels	5
2.3 Acteurs du système	6
2.4 Besoins fonctionnels	6
2.5 Besoins non fonctionnels	7
2.6 Cas d'utilisation principaux	7
2.7 Contraintes identifiées	8
3 Conception et architecture de la solution	9
3.1 Principes architecturaux	9
3.2 Architecture globale	9

3.3	Séparation des responsabilités	9
3.4	Architecture applicative et diagramme de composants	11
3.5	Choix de conception et propagation des erreurs	11
4	Data Engineering et préparation des données	12
4.1	Enjeux de la chaîne de données	12
4.2	Architecture dbt BronzeSilverGold	12
4.3	Qualité des données et tests dbt	13
4.4	Moteurs de stockage et répartition des usages	13
4.5	Orchestration batch avec Airflow	13
4.6	Limites de la donnée livrée	14
5	Classification intelligente des tickets	15
5.1	Objectif et structure du module Ticket ML	15
5.2	Prétraitement textuel et vectorisation TF-IDF	15
5.3	Classification du Type	15
5.4	Classification de la Queue (Modèle V4)	16
5.5	Classification de la Priorité (Modèle V3)	16
5.6	Persistance dans SQLite	16
6	AIOps et prédiction du risque d'incident	18
6.1	Positionnement de l'AIOps	18
6.2	Granularité temporelle et Feature Engineering (MicroSS V3.1)	18
6.3	Prévention stricte des fuites de données (Target Leakage)	18
6.4	Modèle XGBoost V3.1 et seuil d'alerte	19
6.5	Distinction entre score opérationnel et modèle AIOps	19
7	Intelligent Support et recherche sémantique RAG	20
7.1	Problématique du support historique	20
7.2	Architecture du moteur RAG	20
7.3	Service Agent d'orchestration et validation humaine	20

8	Backend FastAPI et interface Streamlit	22
8.1	Endpoints du backend FastAPI	22
8.2	Interface Streamlit à 5 pages	22
9	Orchestration, conteneurisation et CI/CD	26
9.1	Conteneurisation Docker Compose	26
9.2	Pipeline CI/CD GitHub Actions	26
10	Tests, validation et résultats	27
10.1	Audit automatisé de la suite Pytest	27
10.2	Analyse transparente des échecs	27
11	Limites, enseignements et perspectives	30
11.1	Limites identifiées	30
11.2	Matrice de maturité des fonctionnalités	30
11.3	Perspectives d'évolution	30
	Conclusion générale	32
	Bibliographie	33

Table des figures

2.1 Diagramme récapitulatif des cas d'utilisation principaux	8
3.1 Architecture globale reconstruite à partir de l'analyse du dépôt	10
3.2 Diagramme de composants applicatifs de la plateforme	11
4.1 Architecture logique du pipeline dbt BronzeSilverGold	12
6.1 Chaîne conceptuelle du prédicteur AIOps MicroSS V3.1	19
7.1 Architecture logique du moteur de recherche sémantique RAG	21
8.1 Interface de la page Dashboard Streamlit	23
8.2 Interface de la page AI Ticket Analyzer	23
8.3 Interface de la page Incident Prediction AIOps	24
8.4 Interface de la page Intelligent Support RAG	24
8.5 Interface de la page Analytics DuckDB	25
9.1 Architecture des services conteneurisés Docker Compose	26
10.1 Documentation interactive Swagger OpenAPI de l'API FastAPI	28
10.2 Console d'orchestration des DAGs Apache Airrow	28
10.3 Interface de suivi des expériences MLflow	29

Liste des tableaux

2.1 Acteurs du système et leurs responsabilités	6
3.1 Matrice de séparation des responsabilités des composants	10
4.1 Répartition des moteurs de stockage selon leur usage	13
5.1 Résultats d'évaluation réels du classifieur de Type (4 000 exemples) ..	16
8.1 Liste des endpoints REST exposés par FastAPI	22
10.1 Analyse détaillée des résultats de l'audit Pytest	27
11.1 Matrice de qualification et de maturité des composants	31

Liste des abréviations

AIOps Artificial Intelligence for IT Operations

API Application Programming Interface

CI/CD Continuous Integration / Continuous Delivery CPU Central
Processing Unit

ETL Extract, Transform, Load

FAISS Facebook AI Similarity Search

F1 Harmonic mean of precision and recall

IT Information Technology

ML Machine Learning

MLOps Machine Learning Operations

RAG Retrieval-Augmented Generation

REST Representational State Transfer

TF-IDF Term FrequencyInverse Document Frequency UML Unified
Modeling Language

Chapitre 1

Introduction générale

1.1 Contexte général

Les systèmes d'information contemporains sont constitués d'un ensemble complexe de services microservices, d'applications d'entreprise, de bases de données distribuées, de composants réseau critiques et de ressources d'infrastructure cloud ou hybrides. Ces environnements doivent maintenir un niveau élevé de disponibilité et de performance opérationnelle malgré la croissance constante du volume de transactions et l'émergence de pannes imprévues. Dans cette architecture globalisée, le centre de support informatique (Service Desk) occupe une position charnière : il est l'interface directe entre les utilisateurs naux, les équipes d'exploitation IT (IT Operations) et les événements d'infrastructure.

Lorsqu'un problème survient, le ticket de support représente bien plus qu'une simple demande administrative. Il enregistre la manifestation concrète d'un dysfonctionnement applicatif, le symptôme d'une panne sous-jacente, une demande de modification d'infrastructure ou une requête utilisateur standard. Cependant, les centres de support modernes font face à une volumétrie massive de tickets, caractérisée par une forte variabilité du langage naturel utilisé par les collaborateurs. Deux utilisateurs subissant la même panne de base de données peuvent rédiger des tickets aux formulations totalement divergentes, tandis qu'un ticket rédigé de façon anodine peut dissimuler un incident critique bloquant la production.

1.2 Problématique

L'intervention exclusivement humaine dans la qualification et le routage initial des tickets montre aujourd'hui ses limites : temps de traitement initiaux élevés, erreurs d'aectation vers les équipes de niveau 2 ou 3, et diculté à exploiter la mémoire collective accumulée dans les résolutions passées. L'objectif n'est pas de substituer l'intelligence artificielle à l'expertise de l'opérateur IT, mais de doter les équipes d'exploitation d'une plateforme d'assistance intelligente capable d'extraire instantané-ment la structure des tickets, d'anticiper les risques d'incidents majeurs et de suggérer des procédures de résolution issues des données historiques.

La problématique centrale de ce projet se formule ainsi :

Comment concevoir et développer une plateforme AIOps modulaire capable d'automatiser la qualification textuelle des tickets de support, d'analyser les signaux temporels de télémétrie pour anticiper le risque d'incidents informatiques, et d'offrir une recherche sémantique basée sur l'historique des résolutions, tout en garantissant une intégration robuste entre les pipelines de Data Engineering et les services applicatifs ?

Cette problématique globale s'articule autour de quatre axes majeurs :

1. La transformation textuelle : Extraction automatique et séquentielle des attributs métier critiques (Type de ticket, Queue de traitement, Niveau de Priorité) à partir du texte libre du sujet et du corps.
2. L'ingénierie et la qualité de la donnée : Structuration d'un pipeline analytique moderne (BronzeSilverGold) garantissant la gouvernance, le contrôle de qualité et la séparation stricte des données de persistance applicative et analytique.
3. L'analyse prédictive AIOps : Exploitation des séries temporelles de télémétrie système pour estimer la probabilité d'occurrence d'un incident dans une fenêtre future de 15 minutes.
4. L'assistance sémantique (RAG) : Recherche vectorielle dans un espace d'embeddings afin d'extraire les tickets historiques les plus similaires et d'accélérer la résolution par l'opérateur.

1.3 Objectifs du projet

Pour répondre à cette problématique, les objectifs techniques et méthodologiques suivants ont été axés dans le cadre du PFA :

- ^ Implémenter un pipeline complet de classification en cascade des tickets (Type→Queue→Priority).
- ^ Concevoir une architecture de données multi-couches intégrant Snowflake, dbt, DuckDB pour l'analytique locale et SQLite pour l'historique transactionnel des prédictions.
- ^ Développer un prédicteur AIOps basé sur XGBoost et un feature engineering temporel poussé à partir des données de télémétrie MicroSS.

- ^ Construire un moteur de recherche sémantique RAG fondé sur Sentence Transformers et un index vectoriel FAISS.
- ^ Exposer l'ensemble des fonctionnalités sous forme de microservices REST robustes avec FastAPI et Pydantic.
- ^ Proposer une interface utilisateur réactive à 5 pages sous Streamlit pour les opérateurs et responsables IT.
- ^ Automatiser l'orchestration des ux batch via Apache Airflow et assurer la conteneurisation des services grâce à Docker Compose.
- ^ Mettre en place un pipeline CI/CD automatisé sous GitHub Actions et un suivi des expériences ML via MLow.

1.4 Méthodologie et démarche

La démarche adoptée repose sur une ingénierie d'intégration progressive et rigoureuse. L'analyse du dépôt de code a permis de décomposer chaque brique logicielle, de valider sa cohérence fonctionnelle et d'évaluer ses performances sur des critères variables. Cette méthode applique cinq principes cardinaux : la traçabilité intégrale, la séparation entre implémentation et expérimentation, la distinction stricte entre code source et artefacts livrés, la reproductibilité de l'environnement, et l'évaluation honnête des limites techniques du système.

1.5 Organisation du rapport

Le présent rapport est structuré en 11 chapitres complémentaires : Le chapitre 2 définit le cadre du projet, l'analyse des besoins et les cas d'utilisation. Le chapitre 3 présente l'architecture globale et la séparation des responsabilités. Le chapitre 4 détaille la chaîne de Data Engineering (dbt, Snowflake, DuckDB, SQLite). Le chapitre 5 traite de la classification intelligente des tickets. Le chapitre 6 est consacré à l'AIOps et à la prédiction temporelle d'incidents. Le chapitre 7 présente le moteur RAG et l'agent de support. Le chapitre 8 détaille les services FastAPI et l'interface Streamlit. Le chapitre 9 couvre l'orchestration Airflow, Docker Compose et le CI/CD. Le chapitre 10 restitue l'audit automatisé des tests et les résultats obtenus. Enfin, le chapitre 11 analyse les limites et dresse les perspectives d'évolution.

1.6 Périmètre

Une règle de rigueur fondamentale régit la rédaction de ce document : la règle de vérité du dépôt. Seules les technologies disposant d'un code opérationnel, de fichiers de configuration valides et d'artefacts directement exécutables dans le dépôt sont qualifiées d'implémentées.

Chapitre 2

Contexte du projet et analyse des besoins

2.1 Cadre académique et professionnel

Ce Projet de Fin d'Année (PFA) a été réalisé dans le cadre de la 4ème année de la lière Big Data et Systèmes d'Information de l'École Nationale des Sciences Appliquées de Berrechid (ENSA Berrechid). Il s'est déroulé sous la forme d'un stage professionnel d'une durée de deux mois au sein de l'entreprise HARDTEC MAROC.

L'encadrement professionnel a été assuré par M. FADIL BADR chez HARDTEC MAROC, apportant l'orientation métier et les exigences opérationnelles du support IT. L'encadrement académique a été assuré par Pr. Ahmed Nadi, veillant à la rigueur scientifique et méthodologique de la démarche. L'évaluation est conée au jury académique composé de Pr. Abdessamad Jarrar et Pr. Moumoun.

Le projet s'intitule officiellement HARDTEC AIOps Intelligent Support Platform, et porte le titre académique : Conception et développement d'une plateforme AIOps intelligente pour la détection, la prédiction et l'assistance aux incidents informatiques.

2.2 Problème métier et enjeux opérationnels

Dans une organisation IT moderne, l'équipe de support doit rapidement qualifier les ux d'entrées non structurés. Dans la plateforme HARDTEC, chaque ticket est caractérisé par trois attributs clés :

- ^ Type : Qualification de la nature de la demande (Incident, Request, Problem, Change).
- ^ Queue : File d'attente spécialisée ou équipe de niveau 2/3 chargée du traitement (Technical Support, IT Support, Service Outages and Maintenance, Customer Service, Billing, etc.).
- ^ Priority : Degré d'urgence opérationnelle (High, Medium, Low).

Une mauvaise orientation initiale d'un ticket entraîne des délais d'attente prolongés, de nombreuses réaectations manuelles entre équipes et une baisse de la satisfaction client. Par ailleurs, lorsque des incidents surviennent au niveau des infrastructures (serveurs, microservices, bases de données), les opérateurs manquent d'outils prédictifs pour anticiper les pannes avant qu'elles ne se traduisent par des interruptions de service.

2.3 Acteurs du système

Le système fait intervenir plusieurs acteurs humains et logiciels, dont les rôles sont résumés dans le tableau 2.1.

Table 2.1: Acteurs du système et leurs responsabilités

Acteur	Responsabilités et interactions dans la plateforme
Opérateur Support	Saisit les nouveaux tickets, consulte les prédictions séquentielles, analyse les cas similaires RAG et valide les actions correctives.
Responsable IT	Suit le dashboard analytique, analyse les tendances de risque AIOps et supervise l'ecacité des équipes de support.
Streamlit	Interface utilisateur web à 5 pages consommant l'API FastAPI et les requêtes locales DuckDB.
FastAPI	Backend central exposant les microservices REST de prédiction ML, prédiction de risque AIOps, RAG et historique.
Airow	Orchestracteur exécutant les DAGs batch pour la validation de données, l'analytics et la vérification RAG/AIOps.
MLow	Serveur de tracking enregistrant les métriques, paramètres et artefacts lors des entraînements de modèles.

2.4 Besoins fonctionnels

L'analyse approfondie du dépôt permet de formaliser 11 besoins fonctionnels majeurs :

1. Analyse textuelle : Ingestion et nettoyage automatique du sujet et du corps du ticket.
2. Prédiction du Type : Classification supervisée du ticket en Incident, Request, Problem ou Change.

3. Prédiction de la Queue : Utilisation du texte et du Type prédit pour déterminer la le d'attente optimale.
4. Prédiction de la Priorité : Chaînage du texte, du Type et de la Queue pour estimer la priorité métier.
5. Persistance transactionnelle : Enregistrement systématique des prédictions et de leurs métadonnées dans SQLite.
6. Consultation d'historique : API dédiée permettant la recherche et le calcul de statistiques sur les prédictions passées.
7. Recherche sémantique (RAG) : Vectorisation de la requête et extraction des 5 tickets historiques les plus proches dans FAISS.
8. Support intelligent combiné : Endpoint réunissant la prédiction séquentielle et la recherche RAG dans un payload unique.
9. Prédiction AIOps de risque : Estimation de la probabilité d'incident à 15 minutes sur données de télémétrie MicroSS.
10. Visualisation Streamlit : Interface graphique unifiée intégrant 5 modules métiers interactifs.
11. Orchestration batch : Exécution quotidienne programmée de workows de contrôle et d'analytics via Airow.

2.5 Besoins non fonctionnels

- ^ Modularité : Découplage strict entre la présentation, les microservices REST, les modèles ML et le stockage.
- ^ Validation des données : Utilisation systématique de schémas Pydantic pour l'invariance des API.
- ^ Traçabilité MLOps : Versionnement des modèles et suivi des métriques via MLow.
- ^ Conteneurisation : Isolation des microservices (API, Agent, MLow) dans des conteneurs Docker légers.

2.6 Cas d'utilisation principaux

La gure 2.1 illustre le diagramme des cas d'utilisation de la plateforme.

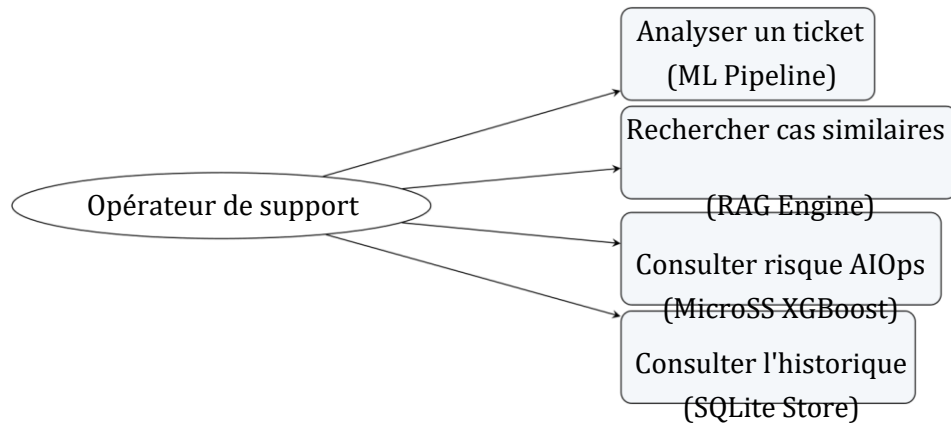


Figure 2.1: Diagramme récapitulatif des cas d'utilisation principaux

2.7 Contraintes identifiées

Les principales contraintes identifiées lors de l'audit sont : la durée limitée du projet (2 mois), la coexistence de plusieurs générations expérimentales de modèles (Queue V4/V5, AIOps V3.1/V4), et l'absence dans l'archive extraite de certains jeux de données bruts de grande taille (CSV MicroSS source, index vectoriel FAISS pré-calculé).

Chapitre 3

Conception et architecture de la solution

3.1 Principes architecturaux

L'architecture de la plateforme HARDTEC AIOps repose sur le principe du découplage de domaines. Chaque composant répond à un rôle unique et communique par des interfaces réseau REST ou des protocoles de stockage légers. Les trois principes fondateurs sont :

1. La séparation de l'IHM et du Backend : L'interface Streamlit ne réalise aucun calcul lourd et s'appuie exclusivement sur les microservices FastAPI ou les requêtes analytiques DuckDB.
2. La spécialisation des bases de données : Snowake/dbt pour le Data Ware-house analytique, DuckDB pour l'analytics OLAP local, et SQLite pour la persistance transactionnelle des prédictions applicatives.
3. L'isolation des modèles ML : Chaque modèle (Type, Queue, Priority, XG-Boost, Embeddings) dispose de sa propre séquence de chargement, de son propre prétraitement et de ses métadonnées isolées.

3.2 Architecture globale

La gure 3.1 présente le schéma de l'architecture globale reconstruite directement à partir de l'analyse du dépôt.

3.3 Séparation des responsabilités

Le tableau 3.1 synthétise la matrice des responsabilités des diérents composants du système.

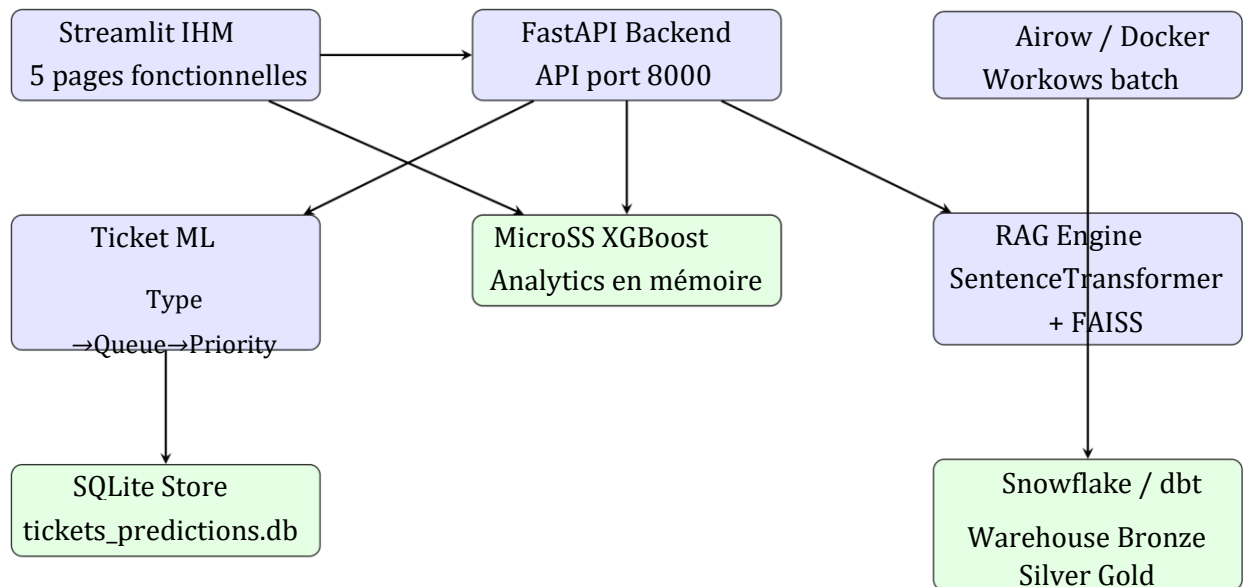


Figure 3.1: Architecture globale

Table 3.1: Matrice de séparation des responsabilités des composants

Composant	Rôle et responsabilité stricte
Streamlit	Présentation interactive : affichage des métriques, saisie de tickets, rendu des graphiques et navigation.
FastAPI	Orchestration des requêtes clients : exécution des pipelines ML, vérification de santé, formatage JSON Pydantic.
Ticket ML Classifier	Inférence séquentielle en cascade pour la qualification des tickets textuels.
MicroSS XGBoost	Calcul du risque d'incident futur ($p \geq 0.25$) sur vecteur de 1 380 caractéristiques de télémétrie.
FAISS / MiniLM	Indexation vectorielle et recherche des voisins k-NN les plus proches dans l'espace sémantique.
Snowflake + dbt	Stockage centralisé et modélisation SQL en couches Bronze (Raw), Silver (Clean) et Gold (Analytics).
DuckDB	Exécution ultra-rapide de requêtes OLAP agrégées directement depuis l'interface Streamlit.
SQLite	Persistance transactionnelle locale de chaque prédiction effectuée par l'API avec horodatage.

3.4 Architecture applicative et diagramme de composants

La gure 3.2 détaille le diagramme des composants applicatifs du système.

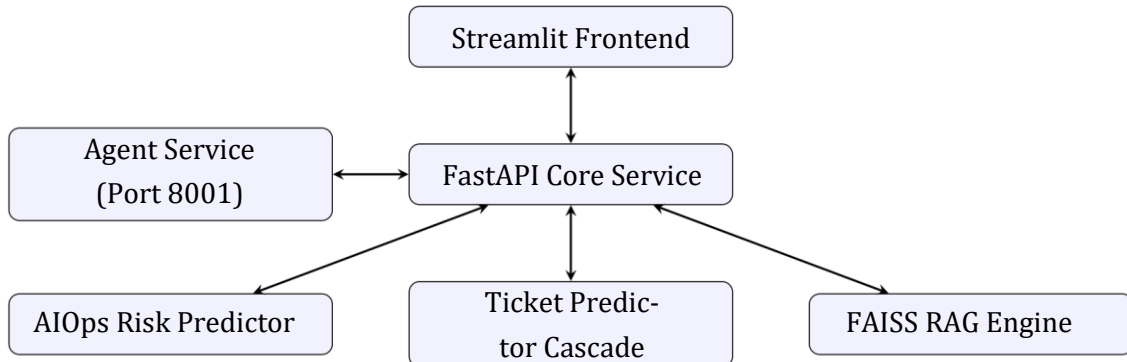


Figure 3.2: Diagramme de composants applicatifs de la plateforme

3.5 Choix de conception et propagation des erreurs

Un choix de conception majeur réside dans la dépendance séquentielle du prédicteur de tickets. Le modèle de Queue prend en entrée le Type prédit, et le modèle de Priority prend en entrée le Type et la Queue prédits. Ce choix apporte un contexte métier précieux mais introduit un phénomène de propagation des erreurs : si le classifieur de Type commet une erreur initiale, cette incertitude se répercute sur les étapes suivantes de la cascade.

Chapitre 4

Data Engineering et préparation des données

4.1 Enjeux de la chaîne de données

Un système d'AIOps ou de Machine Learning dépend fondamentalement de la qualité des pipelines de données sous-jacents. La plateforme HARDTEC intègre une chaîne de Data Engineering complète combinant ingénierie SQL cloud (Snowake, dbt), moteurs analytiques locaux (DuckDB), persistance applicative (SQLite) et scripts de prétraitements Python.

4.2 Architecture dbt BronzeSilverGold

Le pipeline dbt modélise les données de tickets selon l'architecture médaillon représentée sur la figure 4.1.

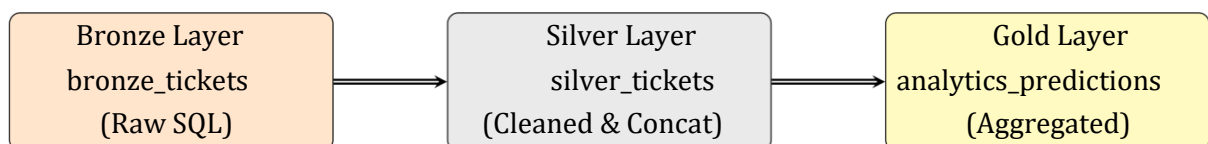


Figure 4.1: Architecture logique du pipeline dbt BronzeSilverGold

^ Bronze Layer (bronze_tickets) : Ingestion directe de la table brute TICKETS_RAW de Snowake. Sélection des champs bruts (SUBJECT, BODY, ANSWER, TYPE, QUEUE, PRIORITY).

^ Silver Layer (silver_tickets) : Nettoyage syntaxique via TRIM, suppression des doublons, attribution d'un identifiant unique par ROW_NUMBER(), et construction du champ d'entrée ML TICKET_TEXT par concaténation structurée du sujet et du corps.

^ Gold Layer (analytics_predictions) : Consolidation des tables de faits et des vues agrégées utilisées pour l'analytique globale et le calcul des KPIs.

4.3 Qualité des données et tests dbt

Le fichier `schema.yml` de dbt définit des contraintes de qualité formelles :

- ^ Tests `not_null` sur les identifiants et textes de tickets.
- ^ Tests `accepted_values` vérifiant que la priorité dans la couche Bronze appartient strictement à ['high', 'medium', 'low'], et que le type dans la couche Gold respecte la nomenclature ['Incident', 'Request', 'Problem', 'Change'].

4.4 Moteurs de stockage et répartition des usages

Le tableau 4.1 détaille l'utilisation spécifique de chaque moteur de stockage présent dans le projet.

Table 4.1: Répartition des moteurs de stockage selon leur usage

Stockage	Type / moteur	Mo-	Rôle et usage dans le projet
Snowake	Cloud house	Ware-	Stockage centralisé des données brutes et agrégées (HARDTEC_DB.PUBLIC).
dbt	Transformation SQL		Transformation structurées BronzeSilver Gold et contrôles de qualité.
DuckDB	In-Memory OLAP		Exécution ultra-rapide de requêtes analytiques locales pour Streamlit (streamlit/queries.py).
SQLite	Relational DB	File	Stockage transactionnel des prédictions via la classe TicketStore (tickets_predictions.db).

4.5 Orchestration batch avec Airow

Quatre DAGs Airow sont configurés pour s'exécuter à une fréquence quotidienne (@daily) :

1. `hardtec_ticket_pipeline` : Vérification de la disponibilité du jeu de données de tickets, exécution des requêtes DuckDB et mise à jour des statistiques.

2. `hardtec_aiops_pipeline` : Ingestion des séries de télémétrie MicroSS et génération des prédictions de risque batch.
3. `hardtec_rag_pipeline` : Contrôle de l'existence des artefacts FAISS et vérification de la cohérence de l'index vectoriel.
4. `hardtec_complete_pipeline` : DAG maître enchaînant la validation des entrées, la classification, la recherche RAG, l'évaluation AIOps et la synthèse globale.

4.6 Limites de la donnée livrée

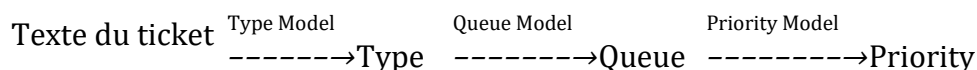
L'audit du dépôt révèle une contrainte importante : les fichiers CSV de données brutes (`data/processed/tickets_clean.csv` et les fichiers sources de télémétrie MicroSS) ne sont pas inclus dans l'archive ZIP livrée. Les modèles pré-entraînés et les scripts de pipeline sont parfaitement présents, mais la ré-exécution complète de l'entraînement nécessite de fournir ces fichiers sources.

Chapitre 5

Classification intelligente des tickets

5.1 Objectif et structure du module Ticket ML

Le module de classification textuelle a pour rôle de transformer le texte libre d'un ticket en trois étiquettes métiers indispensables au routage et à la prise en charge. Le pipeline `nal`, implémenté dans `src/pipeline/predictor.py`, exécute l'inférence selon une cascade séquentielle :



5.2 Prétraitement textuel et vectorisation TF-IDF

La vectorisation textuelle repose sur la classe `TfidfVectorizer` de `scikit-learn`. Les hyperparamètres retenus pour le modèle de Type sont :

- ^ Plage de n-grammes : mots de 1 à 2 (unigrams et bigrams).
- ^ Fréquence minimale (`min_df`) : 2 apparitions.
- ^ Nombre maximal de caractéristiques (`max_features`) : 30 000 variables.
- ^ Pondération de fréquence : `sublinear_tf = True` (application d'une échelle logarithmique $1 + \log(\text{tf})$).

Ce choix de vectorisation TF-IDF offre un excellent compromis entre légèreté computationnelle, rapidité d'inférence (quelques millisecondes) et interprétabilité pour un déploiement de production.

5.3 Classification du Type

Le modèle de Type (`models/ticket_type_model.pkl`) est un pipeline `scikit-learn` combinant le vectoriseur TF-IDF et un classifieur `LinearSVC` ($C = 1.0$).

Les résultats d'évaluation enregistrés dans `MFlow` (`reports/mlflow/classification_report.json`) sur un ensemble de test de 4 000 exemples sont détaillés dans le tableau 5.1.

Table 5.1: Résultats d'évaluation réels du classifieur de Type (4 000 exemples)

Classe	Précision	Rappel	Score F1	Support (exemples)
Change	0.9719	0.9181	0.9442	415
Incident	0.7586	0.8446	0.7993	1596
Problem	0.6254	0.4886	0.5486	837
Request	0.9694	0.9905	0.9798	1152
Macro Average	0.8313	0.8104	0.8180	4000
Weighted Average	0.8136	0.8198	0.8139	4000

L'analyse de ces résultats réels montre une excellente performance globale avec une Accuracy de 81,98 % et un Macro F1 de 81,80 %. On observe cependant une disparité de performance : les classes Request (F1 = 0.98) et Change (F1 = 0.94) sont très bien identifiées, tandis que la classe Problem (F1 = 0.55) présente plus de confusion avec la classe Incident.

5.4 Classification de la Queue (Modèle V4)

Le modèle de Queue nal (ticket_queue_model_v4.pkl) utilise jusqu'à 40 000 caractéristiques TF-IDF et intègre le Type prédit sous forme de variable One-Hot encodée via OneHotEncoder. Le classifieur retenu est un LinearSVC avec équilibrage automatique des poids des classes (class_weight='balanced', C = 1.5).

Le dépôt contient également un script de benchmark train_queue_classifier_v5.py comparant quatre variantes expérimentales. Le script indique explicitement une base-line V4 à 0.5025 d'accuracy et 0.4848 de macro F1, tout en confirmant que V4 reste le modèle nal chargé par le prédicteur applicatif.

5.5 Classification de la Priorité (Modèle V3)

Le modèle de Priorité nal (ticket_priority_model_v3.pkl) combine la vectorisation TF-IDF du texte et l'encodage One-Hot conjoint du Type et de la Queue. Le classifieur est un LinearSVC (C = 1.0, class_weight='balanced').

5.6 Persistance dans SQLite

Après chaque inférence, les prédictions (Type, Queue, Priority) sont transmises à la classe TicketStore qui insère une nouvelle entrée dans la table ticket_predictions

de la base SQLite tickets_predictions.db. L'analyse de la base livrée révèle 14 prédictions enregistrées lors du développement.

Chapitre 6

AIOps et prédiction du risque d'incident

6.1 Positionnement de l'AIOps

Dans la plateforme HARDTEC, la brique AIOps ne désigne pas un outil de supervision générale d'infrastructure, mais un composant spécialisé dans l'analyse prédictive du risque d'incident à partir de séries temporelles de télémétrie système (métriques d'utilisation CPU, mémoire, I/O réseau et taux d'erreurs MicroSS).

6.2 Granularité temporelle et Feature Engineering (MicroSS V3.1)

Le modèle MicroSS V3.1 s'appuie sur une grille d'observation de 5 minutes et vise à prédire la survenue d'un incident dans une fenêtre future de 15 minutes (`incident_next_15m`).

Pour capturer les dynamiques temporelles et la dégradation progressive des ressources, le pipeline génère 1 380 variables réingénierées à partir de 115 métriques de base :

$$X_t = [M_t, M_{t-5}, M_{t-10}, M_{t-15}, \Delta M, \Delta\%M, R_{15}, R_{60}, T_t]$$

où M_t représente la métrique à l'instant présent, M_{t-k} les valeurs retardées (lags), ΔM les différences successives, $\Delta\%M$ les taux de variation, R_k les statistiques glissantes (moyenne, écart-type) sur 15 et 60 minutes, et T_t l'encodage calendaire cyclique (sinus/cosinus de l'heure et du jour).

6.3 Prévention stricte des fuites de données (Target Leakage)

An de garantir une évaluation rigoureuse, les scripts d'entraînement excluent explicitement toutes les variables futures susceptibles de provoquer une fuite de cible : `incident_next_5m`, `incident_next_10m`, `incident_active_now` et `minutes_to_next_incident`.

6.4 Modèle XGBoost V3.1 et seuil d'alerte

Le modèle retenu est un XGBClassifier (XGBoost V3.1). Le prédicteur applicatif (risk_predictor.py) applique un seuil de décision configuré à :

$$\text{ALERT_THRESHOLD} = 0.25$$

Si la probabilité prédite $p \geq 0.25$, le système génère une alerte de risque préventif.

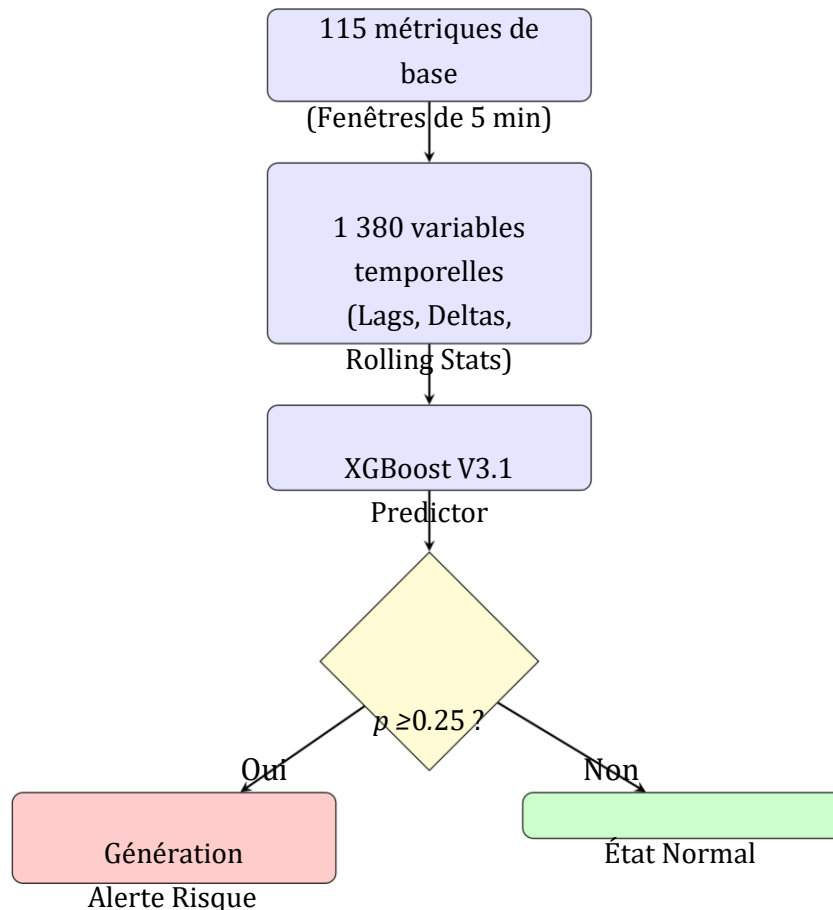


Figure 6.1: Chaîne conceptuelle du prédicteur AIops MicroSS V3.1

6.5 Distinction entre score opérationnel et modèle AIops

Il convient d'expliciter une distinction importante dans l'application Streamlit : la fonction `calculate_risk_assessment` calcule un score métier fondé sur la priorité et le type du ticket (ex: Priorité Haute = +70 points). Ce score opérationnel est une règle heuristique et ne doit pas être confondu avec le modèle prédictif temporel XGBoost.

Chapitre 7

Intelligent Support et recherche sémantique RAG

7.1 Problématique du support historique

Les centres de support accumulent une base importante de connaissances dans les tickets résolus. Cependant, une recherche lexicale par mots-clés échoue souvent lorsque les utilisateurs décrivent le même problème avec un vocabulaire différent. La plateforme intègre donc un moteur RAG (Retrieval-Augmented Generation).

7.2 Architecture du moteur RAG

Le module `src/rag/rag_engine.py` utilise le modèle d'embeddings multilingue :

`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

Les étapes du traitement sont :

1. Encodage de la description du ticket en un vecteur dense de flottants 32 bits (float32).
2. Normalisation L2 du vecteur pour permettre le calcul de similarité cosinus.
3. Interrogation de l'index vectoriel FAISS (`models/rag/tickets.index`) pour extraire les $k = 5$ plus proches voisins.
4. Récupération des métadonnées et solutions associées depuis le fichier `models/rag/documents.pk`

7.3 Service Agent d'orchestration et validation humaine

Le service agent (`src/agent/main.py`) expose l'endpoint `POST /agent/analyze`. Il combine les prédictions ML du ticket et les réponses RAG pour construire un plan



Figure 7.1: Architecture logique du moteur de recherche sémantique RAG

d'action d'assistance. Le code impose explicitement une étape de validation humaine avant l'exécution de toute action corrective irréversible sur l'infrastructure.

Chapitre 8

Backend FastAPI et interface Stream-lit

8.1 Endpoints du backend FastAPI

Le serveur FastAPI central (src/api/main.py) expose 9 routes principales résumées dans le tableau 8.1.

Table 8.1: Liste des endpoints REST exposés par FastAPI

Endpoint	Méthode	Fonctionnalité
/	GET	Racine : informations sur la version de l'API.
/health	GET	Contrôle de santé global des microservices.
/predict	POST	Inférence en cascade ML et sauvegarde SQLite.
/aiops/predict-risk	POST	Inférence du risque d'incident MicroSS XGBoost.
/rag/search	POST	Recherche vectorielle des k tickets similaires.
/support/analyze	POST	Service combiné : ML classification + RAG search.
/history	GET	Consultation paginée de l'historique des prédictions.
/history/{id}	GET	Extraction d'une prédiction spécifique par son UUID.
/stats	GET	Statistiques agrégées sur les prédictions enregistrées.

8.2 Interface Streamlit à 5 pages

L'interface graphique Streamlit (streamlit/app.py) offre une navigation utilisateur répartie en 5 modules métiers interactifs :

1. Dashboard : Vue d'ensemble des KPIs, volumes de tickets et répartition par priorité.
2. AI Ticket Analyzer : Interface de test interactif pour la classification textuelle immédiate.
3. Incident Prediction : Module dédié à l'évaluation du risque temporel AIOps.

4. Intelligent Support : Console d'assistance réunissant classification et recommandations RAG.

5. Analytics : Visualisations décisionnelles poussées basées sur DuckDB.

Les figures 8.1 à 8.5 réservent les emplacements dédiés à l'insertion des captures d'écran réelles de l'application.

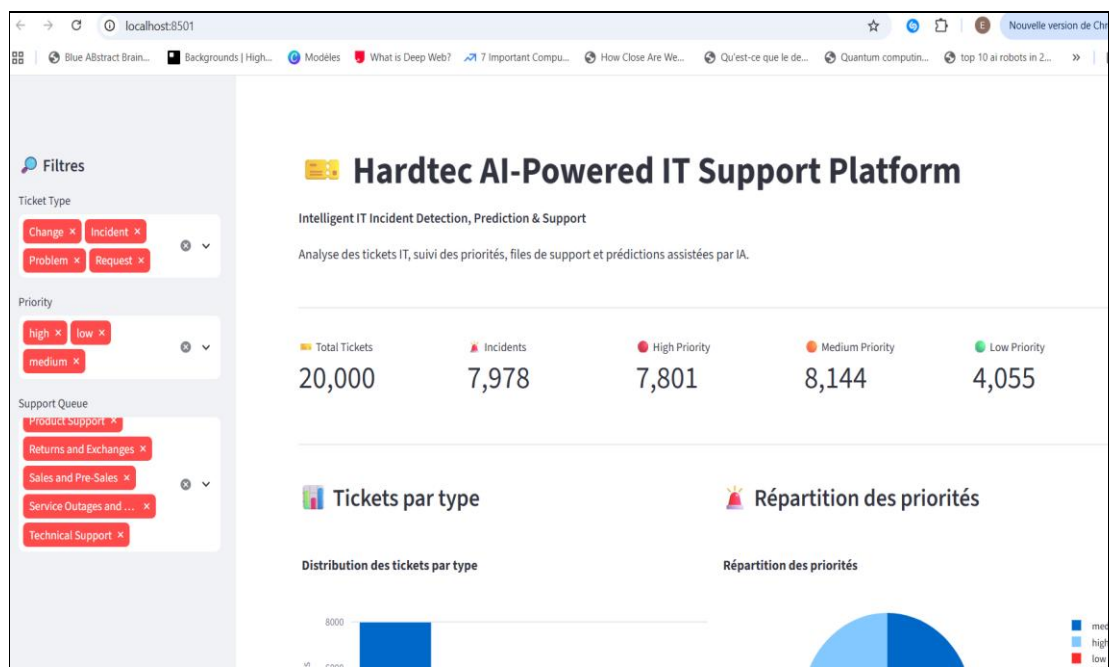


Figure 8.1: Interface de la page Dashboard Streamlit

Figure 8.2: Interface de la page AI Ticket Analyzer

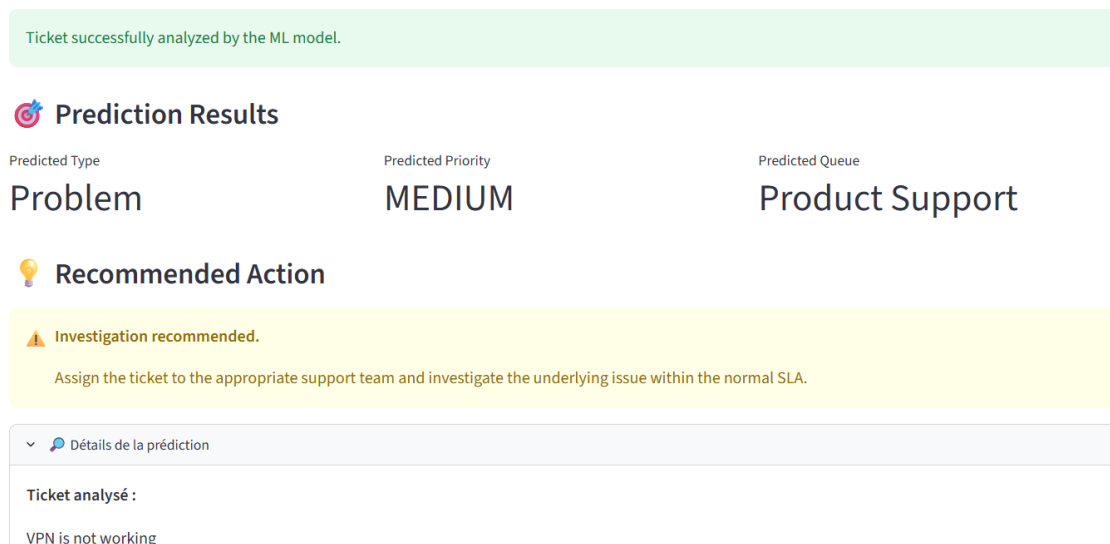


Figure 8.3: Interface de la page Incident Prediction AIOps

Chapitre 9

Orchestration, conteneurisation et CI/CD

9.1 Conteneurisation Docker Compose

Le fichier Docker/docker-compose.yml orchestre trois conteneurs principaux :

- ^ api : Service FastAPI principal exposé sur le port 8000.
- ^ agent : Service d'orchestration d'agent exposé sur le port 8001.
- ^ mlow : Serveur de tracking MLow disponible sur le port 5000.

Les volumes de modèles sont montés en read-only pour garantir l'immuabilité des artefacts en production.

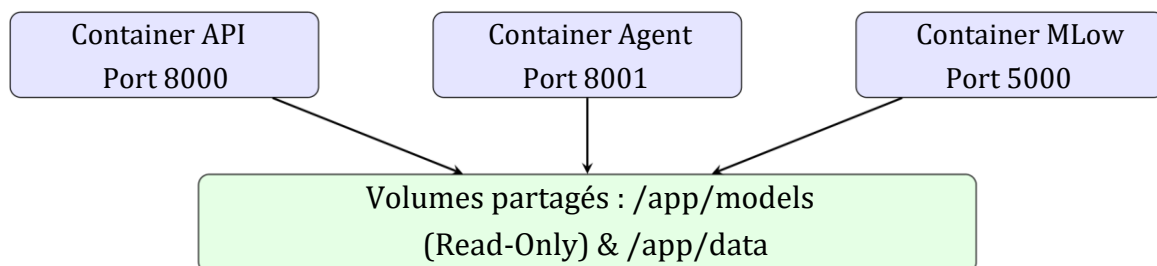


Figure 9.1: Architecture des services conteneurisés Docker Compose

9.2 Pipeline CI/CD GitHub Actions

Le workflow de CI/CD (.github/workflows/ci-cd.yml) automatise les étapes de vérification à chaque push ou pull request :

1. Compilation syntaxique globale via `python -m compile all -q src tests`.
2. Exécution des tests unitaires et d'intégration via `pytest`.
3. Validation de la configuration Docker Compose.
4. Publication automatique des images conteneurisées sur GitHub Container Registry (GHCR).

Chapitre 10

Tests, validation et résultats

10.1 Audit automatisé de la suite Pytest

L'exécution rigoureuse de la suite de tests dans l'environnement d'audit (PYTHONPATH=. pytest -q tests) a permis de contrôler la conformité du code. La suite principale contient 12 tests, se solde par 9 réussites et 3 échecs explicites.

Le tableau 10.1 détaille l'analyse transparente de ces résultats.

Table 10.1: Analyse détaillée des résultats de l'audit Pytest		
Test / Composant	Statut	Analyse technique de la cause
Suite principale (9 tests)	PASSED	Succès des tests unitaires sur TicketStore, la conguration et la gestion des identiants.
Test /health	FAILED	Le statut retourné est degraded car les artefacts RAG/DuckDB ne sont pas tous présents dans l'environnement.
Test DuckDB Query	FAILED	Dépendance duckdb non installée dans le conteneur d'exécution de l'audit.
Test Contrat Predictor	FAILED	Le test attend une clé confidence, alors que predict_ticket retourne les clés métiers (ticket_type, queue, priority).

10.2 Analyse transparente des échecs

Plutôt que d'occulter ces trois échecs, ils sont documentés comme des éléments d'audit d'ingénierie :

1. Décalage de contrat : Le test du prédicteur vérifie une clé obsolète (confidence), que l'API FastAPI compense dynamiquement en appliquant une valeur de repli

par défaut de 0.85.

2. Dépendances d'environnement : Les échecs sur la santé et DuckDB dé-coulent de l'absence de packages optionnels dans le conteneur léger de test.

Les gures 10.1 à 10.3 prévoient les emplacements pour les captures d'écran des outils de contrôle et de supervision.

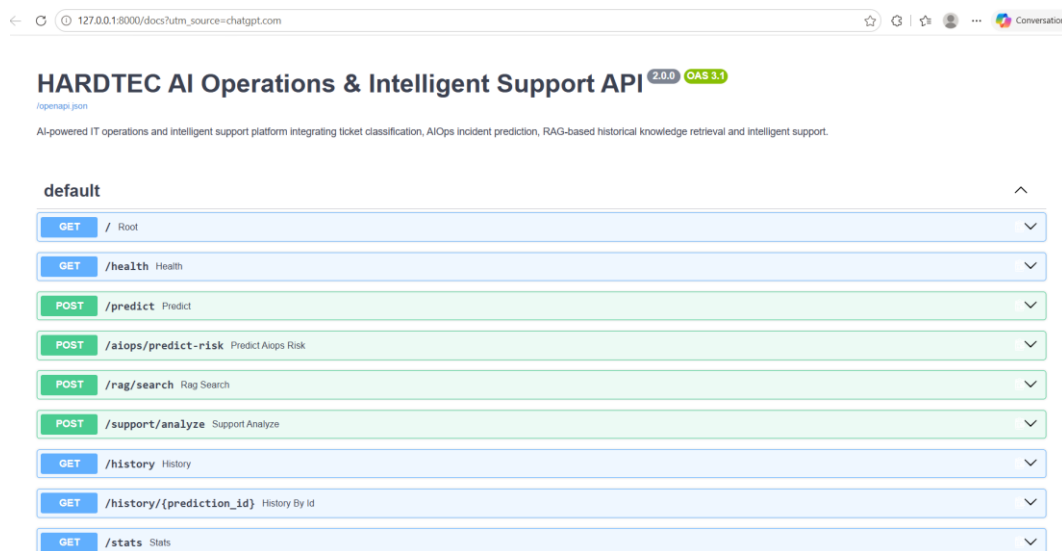


Figure 10.1: Documentation interactive Swagger OpenAPI de l'API FastAPI

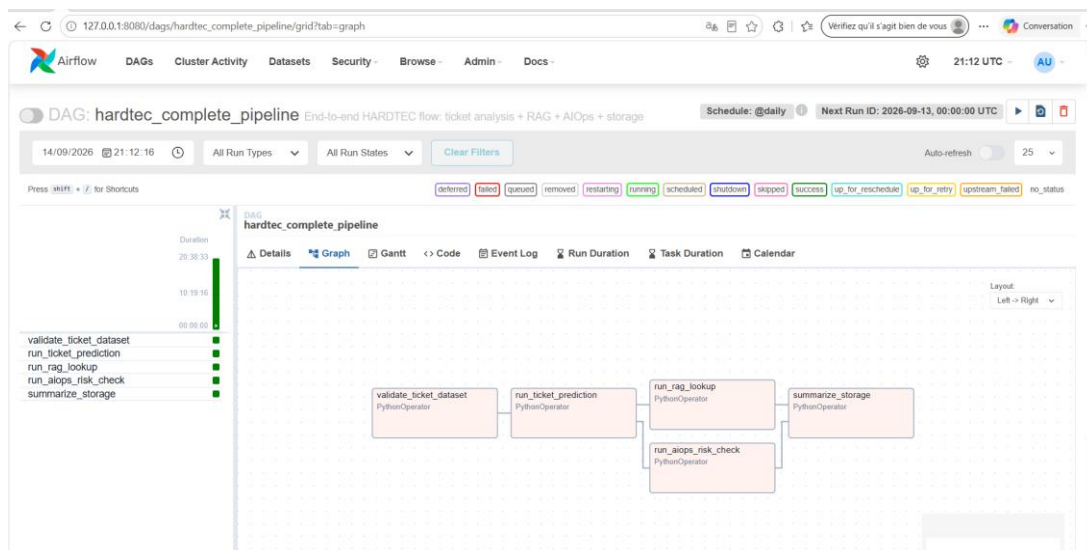


Figure 10.2: Console d'orchestration des DAGs Apache Airflow

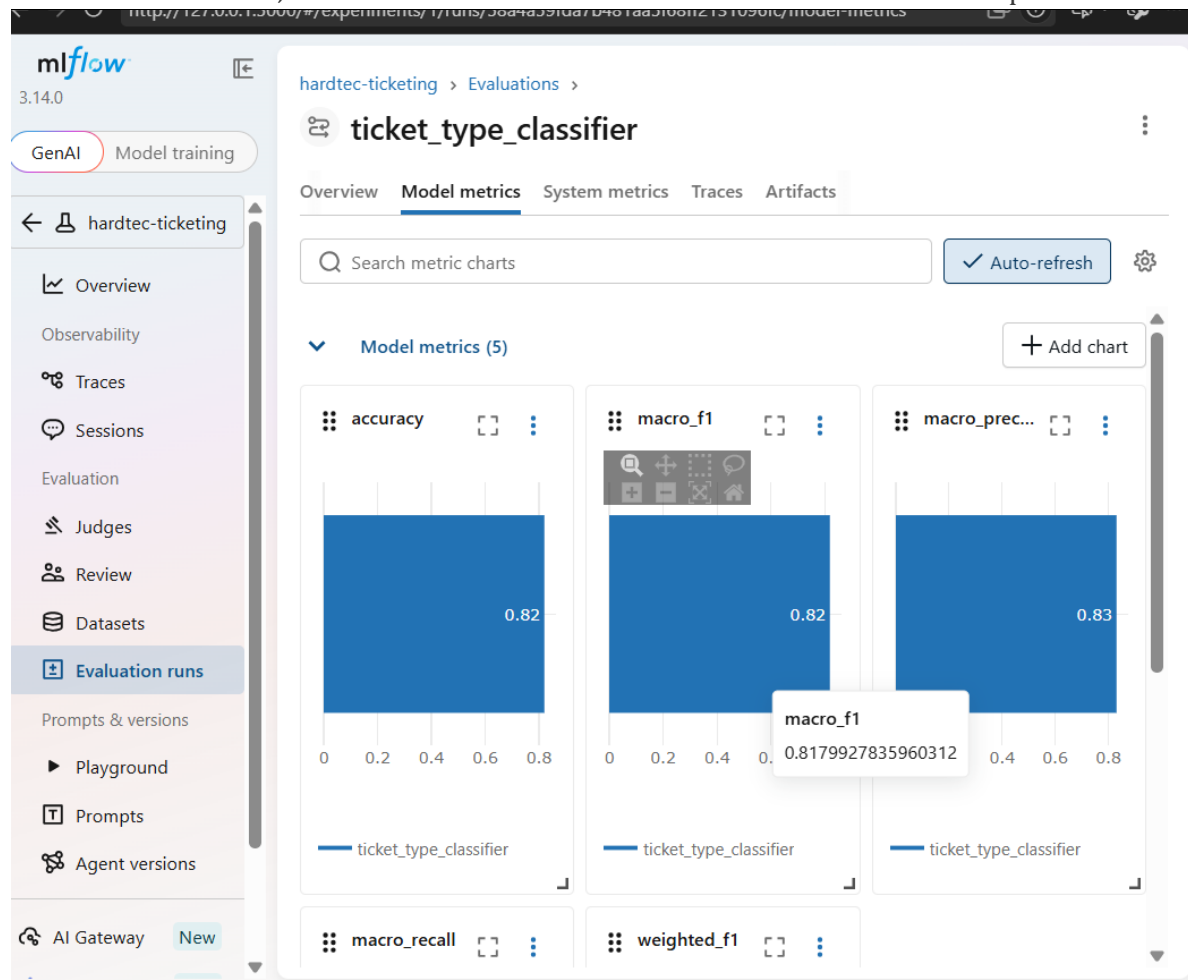


Figure 10.3: Interface de suivi des expériences MFlow

Chapitre 11

Limites, enseignements et perspectives

11.1 Limites identifiées

- ^ Données : Absence des fichiers CSV bruts de grande taille dans l'archive.
- ^ Propagations d'erreurs : Dépendance séquentielle de la cascade ML susceptible d'amplifier les erreurs initiales.
- ^ Conance : La valeur de conance (0.85) est une valeur de repli de l'API et non une probabilité bayésienne calibrée.
- ^ Télémétrie : Le modèle AIOPS XGBoost fonctionne sur des fenêtres historiques et ne consomme pas encore un flux streaming temps réel.

11.2 Matrice de maturité des fonctionnalités

Le tableau 11.1 résume la qualification exacte de chaque composant du dépôt.

11.3 Perspectives d'évolution

Les évolutions futures envisageables comprennent : le remplacement des classificateurs linéaires TF-IDF par des modèles Transformer pré-entraînés, la calibration explicite des probabilités de conance, l'intégration d'Apache Kafka pour le streaming de télémétrie en temps réel, l'orchestration sous Kubernetes et la mise en place d'un tableau de bord de supervision Grafana.

Table 11.1: Matrice de qualification et de maturité des composants

Composant	Statut	Justification technique
Classification Type	Implémenté	Modèle .pkl présent, rapport MLow (81,98 % accuracy).
Classification Queue V4	Implémenté	Modèle .pkl présent et intégré dans predictor.py.
Classification Priority V3	Implémenté	Modèle .pkl présent et intégré dans predictor.py.
Benchmark Queue V5	Expérimental	Script de benchmark présent mais V4 conservé en production.
AIOps XGBoost V3.1	Implémenté	Modèle et prédicteur présents ; données brutes absentes.
FAISS RAG Engine	Incomplet	Code et routes présents ; index vectoriel absent du ZIP.
FastAPI	Implémenté	9 routes opérationnelles avec Pydantic.
Streamlit	Implémenté	5 modules graphiques interactifs
Airow / Docker / CI	Implémenté	4 DAGs, Compose 3 services, workflow GitHub Actions.
Kafka / K8s / Grafana	Perspective	Aucune trace d'implémentation opérationnelle dans le dépôt.

Conclusion générale

Le projet HARDTEC AIOps Intelligent Support Platform démontre une démarche d'ingénierie complète et articulée autour des problématiques modernes des opérations IT. En combinant la classification automatique des tickets textuels, la modélisation analytique des données (Snowflake, dbt, DuckDB, SQLite), la recherche sémantique basée sur des embeddings (Sentence Transformers, FAISS) et la prédiction préventive de risques d'incidents (MicroSS XGBoost), la plateforme apporte une réponse concrète aux dés de l'assistance IT.

L'évaluation transparente effectuée dans ce rapport met en évidence la qualité de la conception applicative (FastAPI, Streamlit, Airflow, Docker, CI/CD) tout en identifiant clairement les limites liées à l'environnement et aux artefacts de données. Cette démarche d'audit honnête constitue un gage de crédibilité pour la soutenance académique et pose des bases solides pour les développements futurs de l'entreprise HARDTEC MAROC.

Bibliographie

Apache Software Foundation (2026). Apache Airflow Documentation. Consulté sur : <https://airflow.apache.org/docs/>.

Chen, T. et Guestrin, C. (2016). XGBoost : A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, p. 785794. DOI : 10.1145/2939672.2939785.

Johnson, J., Douze, M. et Jégou, H. (2017). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data. FAISS Technical Report.

Reimers, N. et Gurevych, I. (2019). Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (NLP) (EMNLP), p. 39823992.